

Coverage-based regression test case selection, minimization and prioritization: a case study on an industrial system

Daniel Di Nardo^{1,*}, Nadia Alshahwan¹, Lionel Briand¹ and Yvan Labiche²

¹*Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg, Luxembourg*

²*Systems and Computer Engineering, Carleton University, Ottawa, Ontario, Canada*

SUMMARY

This paper presents a case study of coverage-based regression testing techniques on a real world industrial system with real regression faults. The study evaluates four common prioritization techniques, a test selection technique, a test suite minimization technique and a hybrid approach that combines selection and minimization. The study also examines the effects of using various coverage criteria on the effectiveness of the studied approaches. The results show that prioritization techniques that are based on additional coverage with finer grained coverage criteria perform significantly better in fault detection rates. The study also reveals that using modification information in prioritization techniques does not significantly enhance fault detection rates. The results show that test selection does not provide significant savings in execution cost (<2%), which might be attributed to the nature of the changes made to the system. Test suite minimization using finer grained coverage criteria could provide significant savings in execution cost (79.5%) while maintaining a fault detection capability level above 70%, thus representing a possible trade-off. The hybrid technique did not provide a significant improvement over traditional minimization techniques. Copyright © 2015 John Wiley & Sons, Ltd.

Received 26 July 2013; Revised 7 January 2015; Accepted 23 January 2015

KEY WORDS: regression testing; industrial case study; test case selection; test suite minimization; test case prioritization

1. INTRODUCTION

Regression test selection, minimization and prioritization have been a major research topic with hundreds of papers that investigate the different techniques. As further described in Section 2, various techniques for test case selection, prioritization, and minimization have been defined and studied to help cope with the complexity of testing large systems. Typically, test suites grow as software systems evolve over subsequent versions. Testing can be a very time-consuming and costly undertaking, and approaches to reduce regression testing costs are highly desirable.

Many approaches for implementing regression testing strategies have been proposed. A large body of existing work focuses on coverage-based strategies. In the case of test case selection, code coverage information along with code modification information is used to determine what test cases cover code that has changed between software versions; this data is then used to select a subset of the original test suite for regression testing on the next version. Test suite minimization strategies use code coverage information recorded against an initial version of software to determine what tests to run on a subsequent version. Prioritization strategies seek to order test cases based on code coverage and the analysis of code modifications. Coverage-based strategies with or without the use

*Correspondence to: Daniel Di Nardo, Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg, Luxembourg.

†E-mail: daniel.dinardo@uni.lu

of code modification information have the advantage that they can be fully automated without the use of additional inputs.

As further discussed in Section 7, there are few industrial case studies involving real faults and assessing coverage-based regression testing techniques. These studies are not in any way complete and, in any case, must be replicated for converging towards reliable results that can be used by industry for deciding about their regression testing approach. There are other studies involving small systems and seeded faults or mutants. Though they are useful, realistic studies in industrial contexts are required.

This paper reports on a carefully designed case study involving a complex industrial system with multiple versions and real regression faults. It also carefully compares the results of the case study with existing studies and attempts to provide general conclusions based on all the existing evidence. The main goal of this study is to evaluate and compare coverage-based regression testing techniques: regression test case selection, test suite minimization, a hybrid selection with minimization approach and test case prioritization (total coverage versus additional coverage with and without the use of modification information). This paper is an extension of a previous case study that focused only on coverage-based prioritization techniques [1]. To the best of the authors' knowledge, this study is the most comprehensive investigation of regression testing techniques (seven approaches using five coverage levels) on the same industrial system, thereby facilitating comparisons and, hopefully, a deeper insight into the studied techniques.

The results of this case study show that when applying test case selection, almost no savings in test suite size are realized. The study also reveals that applying test suite minimization greatly reduces execution cost, but with a notable reduction in fault detection effectiveness.

The results reveal that prioritization techniques based on additional coverage perform better than those based on total coverage. The case study also reveals that using modification information does not improve fault detection rates of the prioritization techniques.

Furthermore, this study investigates the effect of using five different structural coverage criteria on the effectiveness of the different approaches. The results show that across all techniques, finer grained coverage criteria perform better than coarser grained coverage criteria when a difference in effectiveness can be observed.

Rigorous comparisons with existing studies show that some of this study's results are consistent with previous findings while others are contradictory, suggesting that the conditions under which regression testing strategies perform well must be better understood.

The rest of this paper is organized as follows: Section 2 provides a more detailed background on regression testing approaches. Section 3 explains the design of the case study, whilst Section 4 describes data collection and analysis. Section 5 presents and discusses the results of the case study. Section 6 examines threats to the validity of this study. Section 7 reports on related work and compares existing results with those of this study, in order to draw more general conclusions regarding coverage-based regression testing. Section 8 concludes this study.

2. REGRESSION TESTING METHODOLOGIES

Regression testing is a testing activity that is performed to validate that changes to the code while a system evolves do not affect existing unchanged functionality. With no time or resource restrictions, the tester would ideally re-execute all test cases that were successfully executed on the previous version of the system to perform regression testing. This is known as the 'Retest-All' strategy. However, as the test suite grows with each new version, re-executing all test cases might be time-consuming and, in some cases, infeasible. Therefore, several regression testing strategies have been proposed in the literature: test suite minimization, test case selection and test case prioritization.

As the system under test evolves, its expected behaviour might change, causing some existing test cases to fail. The failure of these test cases does not indicate a fault in the system but rather reflects the changes that were introduced. Therefore, each existing test case must be revalidated before it can be used to test the next version of the system, to exclude any obsolete test cases.

When using the different regression testing techniques, care must be taken to assure that any test interdependencies be considered (e.g. a test case might require that another test case be run

first for proper execution). Regression testing techniques might not be practical should test case dependencies be so complex that a regression testing tool cannot incorporate this information.

The next few sections discuss the regression testing strategies in more detail. Section 2.1 discusses test case prioritization, Section 2.2 provides a background on test case selection, while Section 2.3 discusses test suite minimization.

2.1. Test case prioritization

Test case prioritization aims to order the execution of test cases in the regression test suite, such that the benefit to the tester is maximized. In other words, the test suite is ordered based on some criterion that is expected to lead to the early detection of faults. This prioritization criterion can be based on coverage, requirements, models or the history of modifications to the system. More details about these and other prioritization techniques can be found in Yoo and Harman's survey of regression testing [2]. Prioritization based on coverage is based on the hypothesis that early maximization of coverage could lead to early detection of faults.

Several coverage-based prioritization techniques have been proposed and evaluated in the literature [2]. This paper focuses on four main strategies of coverage-based techniques that have been widely used in previous work:

Total coverage: The total coverage technique prioritizes test cases based on their total individual coverage. That is, test cases that cover more parts of the system, based on a chosen coverage criterion, are ordered before those that cover less. If several test cases have the same coverage, the order is determined randomly.

Additional coverage: The additional coverage technique orders test cases so that coverage is maximized as early as possible. The technique differs from the total coverage approach in that the next test case is selected based on the highest coverage of uncovered code rather than the highest coverage of all code.

Total coverage of modified code: This approach is similar to the total coverage approach, but rather than prioritizing test cases based on highest total coverage, the approach only takes into account coverage of modified code. To apply this approach, the two versions of the system need to be analyzed to identify code modifications.

Additional coverage of modified code: This approach is similar to the additional coverage approach, but like the total coverage of modified code approach, only takes into account modified code.

These four approaches can be applied using different structural coverage criteria. For example, the total coverage approach can be based on the total coverage of function calls, statements or branches. For additional coverage approaches, when all code has already been covered, any remaining test cases must still be ordered. This can be done, for example, by resetting all coverage information to 'not covered' then repeating the same approach on the remaining test cases. For approaches that rely on modified code, coarser grained coverage criteria might result in some test cases that cover changes to not be prioritized earlier in the test sequence. This can happen, for example, if function call coverage criterion is used to prioritize, and a change to the code is made to a global variable. However, the finer grained coverage criteria, such as block and decision coverage, would capture this change.

2.2. Test case selection

Test case selection techniques use information about the old and new versions of the system together with information about the regression test suite to select a subset of test cases to test the new version. Coverage-based selection techniques seek to identify modified program components and select the test cases in the regression test suite that exercise these components.

Rothermel and Harrold [3] provided a summary of code-based regression test selection techniques. The reductions in test suite size for test case selection techniques reported in the literature can be very significant (up to 98%) [4–7]. However, in one study [7], reductions as low as 0% were reported.

Regression test selection techniques can be either safe or unsafe: safe techniques select every test case in the regression test suite that can reveal one or more faults in the new version of the system.

That is, the technique selects every test case that is modification-traversing. Unsafe techniques may discard several test cases from the regression test suite that are modification-traversing; these techniques can often rival the efficacy of the safe selection techniques while dramatically reducing the number of tests that need to be executed [8]. In more critical systems, one may choose a safe technique even though this might be a more time consuming and costly choice; if there is little time to test, one may use an unsafe technique even though this technique may discard fault-revealing test cases [3].

2.3. Test suite minimization

As systems evolve, their test suites will be modified to accommodate new functionality. It is possible that redundant test cases (e.g. test cases that cover system components that are already covered by existing test cases) will be introduced as a test suite grows. Test suite minimization techniques address this problem by seeking to permanently remove redundant test cases in a test suite. The goal is to create a test suite that is more efficient, that is, smaller in size but effective at finding faults. The minimization process is typically accomplished without knowledge of the changes in the new version of the program [9].

Test suite minimization aims to reduce the number of test cases but might also reduce the fault finding capability of the test suite. Wong *et al.* [10] conducted a series of minimization experiments and found that the loss in fault detection was not very significant (2–7%). However, Elbaum *et al.* [11] also conducted minimization experiments, but found that the loss of fault detection capability was very significant (up to 60%).

3. CASE STUDY DESIGN

This section describes the design of the case study: the research questions it aims to answer, the system under study and the experimental procedures.

3.1. Research questions

The case study was designed to answer the following research questions:

RQ1: How does the choice of test case prioritization technique and granularity of coverage criteria used in the analysis affect the rate of fault detection?

Techniques involving change analysis come at a higher cost, and it is therefore useful to find out whether they bring significant benefits. It is also impossible to analytically determine which level of coverage granularity to use and therefore this must be determined experimentally. Furthermore, coarser granularity coverage criteria are more likely to scale to very large systems and should be favoured, unless significant benefits can be demonstrated for finer grained criteria. To answer this question, this study applies test suite prioritization techniques for five different coverage criteria: function entry, function return, block, basic block and decision coverage. This study also compares the fault detection rates of test suites prioritized using several techniques: total coverage, additional coverage, total coverage of modified code and additional coverage of modified code. It then compares the effectiveness of test suites, as discussed in Section 4.4.

RQ2: Can coverage-based test case prioritization improve the rate of fault detection of a regression test suite?

To answer this question, this study compares the fault detection rates of the test suite in its original order, using random ordering and using the best performing coverage-based test case prioritization technique. Techniques based on coverage analysis come at an additional cost; to be justified, they must enable earlier fault detection than just random orders. This study also provides a comparison to the test suite in its optimal order, such that faults are detected the earliest possible. Note that the optimal order can only be determined after the test cases are executed. Therefore, the optimal order is a theoretical maximum bound for prioritization techniques to be compared against and thus facilitates the interpretation of results.

RQ3: Can test case selection techniques reduce the cost of executing a test suite and what is their effect on the fault detection effectiveness of a test suite?

Applying test case selection techniques requires additional analysis of the changes in the source code and the coverage of the test cases. Therefore, the result of applying such techniques should provide a significant reduction in the test suite size (i.e. execution cost) that justifies this additional effort. Moreover, test case selection techniques not only could reduce the execution cost of a test suite but might also affect its fault detection capabilities. Any reduction in the execution cost that test case selection techniques can provide should be weighed against any negative effect this reduction might have on the fault detection capability of the test suite. To answer this question, a coverage-based test selection technique (using different levels of coverage granularity) is applied to the case study system. This study measures and compares the reduction in test suite size for the different coverage criteria and also compares the number of faults detected by the test suites after applying test selection to the number of faults detected by the original test suite.

RQ4: Can test suite minimization techniques reduce the cost of executing a test suite and what is their effect on the fault detection effectiveness of a test suite?

Test suite minimization techniques also require additional analysis and, therefore, should provide a significant reduction in the cost of executing the test suite that offsets this additional effort. Similar to test case selection, if test suite minimization produces reductions in test suite size, these reductions have to be evaluated against the loss of fault detection capability that might result from applying minimization. To answer this question, this study measures and compares the reduction in the test suite size when applying test suite minimization and the number of faults detected compared with the original test suite, using the five different coverage criteria.

RQ5: How does a hybrid approach that combines test case selection and test suite minimization perform compared with the individual approaches in terms of cost reduction and fault detection effectiveness?

The test case selection technique that this study investigates selects all test cases that cover modified code, even if some of these test cases are redundant. Removing these redundant test cases might prove to be effective in reducing cost while maintaining fault detection effectiveness. To answer this question, this study compares the reduction in test suite size and the fault detection effectiveness of test case selection and test suite minimization to a hybrid approach that combines the two approaches for the five coverage criteria. This study also compares the number of faults detected by the hybrid approach to the number of faults detected by the original test suite.

3.2. System under study

For confidentiality reasons, the actual name and version numbers of the industrial software are not used. When describing the system (hereafter referred to as the NoiseGen system), some of the terminology has been changed from that of the original.

The NoiseGen system is designed for digital subscriber line (DSL) technology manufacturers to test the performance of their products by simulating the impairments found on twisted-pair copper wires. The software and hardware are bundled together in a rack-mount PC running the Windows operating system. All versions of the NoiseGen software are implemented in C and C++. Much of the C code has been generated from MATLAB (The MathWorks, Inc., Natick, Massachusetts, USA) code. As well as a graphical user interface (GUI) mode, the system provides a scripting mode that was used in this study to automate the execution of test cases.

Table I. NoiseGen system changes over the five versions used in the study.

Version	LoC	Function Entries		Function Returns		Blocks		Basic Blocks		Decisions	
		Total	Modified	Total	Modified	Total	Modified	Total	Modified	Total	Modified
1	61 615	792	–	1066	–	15 838	–	40 853	–	8023	–
2	59 132	1539	642	1988	692	15 085	14 292	29 750	38 998	9467	6697
3	61 090	1562	605	2018	426	15 662	2610	30 657	3878	10 039	2176
4	67 391	1812	637	2365	454	17 251	3851	32 860	6525	11 366	3121
5	72 925	1848	565	2415	436	18 762	4273	36 766	7301	12 134	3514

Table I provides an overview of the evolution of the system over five consecutive versions selected for this study. For each version, the number of function entries, function returns, blocks, basic blocks and decisions are listed together with the number of those affected by changes to the system in that specific version. The changes considered in the table are modifications and deletions while additions are not included in the count, as they are expected to represent new functionality. For each version, at least four main components were changed. The number of functions affected by the change (as seen in Table I) indicates that the changes are distributed over the system.

Changes were typically made to the core components and application program interface (API) of the system to accommodate new functionalities. Other modifications are made throughout the versions to correct problems. Major architectural changes were made in version 2 (as can be seen in the number of items changed in the table) because of additional GUI functionalities that were added to the system.

3.3. Case study design

This study uses the four coverage-based prioritization strategies described in Section 2.1 because they are the most commonly used in the literature. It also uses a coverage-based test selection technique that analyzes the changes in the new version of the system and then selects all test cases that cover changed or deleted code. For added code, the technique selects test cases that execute the line of code that is just before and after the corresponding position of the added code in the new release. This study uses a heuristic-based minimization technique that minimizes a test suite such that a minimal subset is found that preserves code coverage with respect to the specified code coverage criteria. As mentioned before, to apply the hybrid technique, one first applies the test selection technique and then applies test suite minimization based on the code coverage of the modified code. A technique combining prioritization and minimization has also been proposed in the literature [12], but its evaluation falls outside the scope of this study.

This study uses five different coverage criteria: function entry, function return, block, basic block and decision. Function entry, block and decision coverage levels have all been used in previous work on regression testing [11, 13–17]. Function return and basic block have also been selected to provide additional data points to better understand results and to include all coverage criteria supported by the regression testing tool that was selected to conduct this case study.

A test suite that satisfies function return coverage ensures all explicit or implicit returns or exits from a function are executed at least once. To satisfy basic block coverage, a test suite must execute all code fragments not containing control flow branching or function calls. This is in contrast to traditional block coverage, where only control flow branching is considered. Basic block coverage is provided, in addition to block coverage, because it is possible that a function call will not return.

4. DATA COLLECTION AND ANALYSIS

This section describes the tools and techniques used to collect and analyze data in this study. Section 4.1 provides a brief description of the regression testing tool used, while Sections 4.2 and 4.3 explain the process of generating the test suite and identifying faults. Section 4.4 defines the effectiveness measures used to compare techniques. Finally, Sections 4.5 and 4.6 describe how the experiments were conducted and the methods used to analyze results.

4.1. The xSuds tool

The NoiseGen system runs on the Windows platform. Therefore, the Telcordia Visualization and Analysis Toolsuite (xSuds) [18] that also runs on Windows and is industry-strength was chosen for the experimental study.

The xSuds tool was chosen because it supports the prioritization, selection and minimization strategies selected for this study. The tool's implementation of the techniques matches the description in Sections 2 and 3.3. The tool has also been previously used in regression testing research [10, 19]. The tool can be used with C and C++ code (the programming languages used in the

NoiseGen system) and allows code coverage analysis to be performed for various code coverage criteria. Five of those coverage criteria, which are used for this study, are supported for both C and C++.

4.2. Test cases

The test cases for this study were created after the release of all of the software versions used in this study. This is because the NoiseGen engineers did not maintain a test repository for the system and relied on informal manual testing guided by the specifications of the system.

The test suites were created by following a process based on systematic state-of-the-art practices. Test cases for the different versions of the system were generated by following the well-known black-box category-partition strategy [20]. The category-partition strategy requires the tester to analyze system specifications to identify functional testing units, their parameters and interactions with the environment and derive categories (i.e. test relevant properties). The resulting categories are then partitioned based on expected significant differences in system behaviour, as well as constraints and dependencies between different parameter values and environment states. Test frames are then generated using all possible combinations of input parameters and environment settings subject to the constraints.

Code coverage information was then used to identify uncovered areas of code and guide the creation of additional test cases. Marick's advice [21] was followed such that, rather than

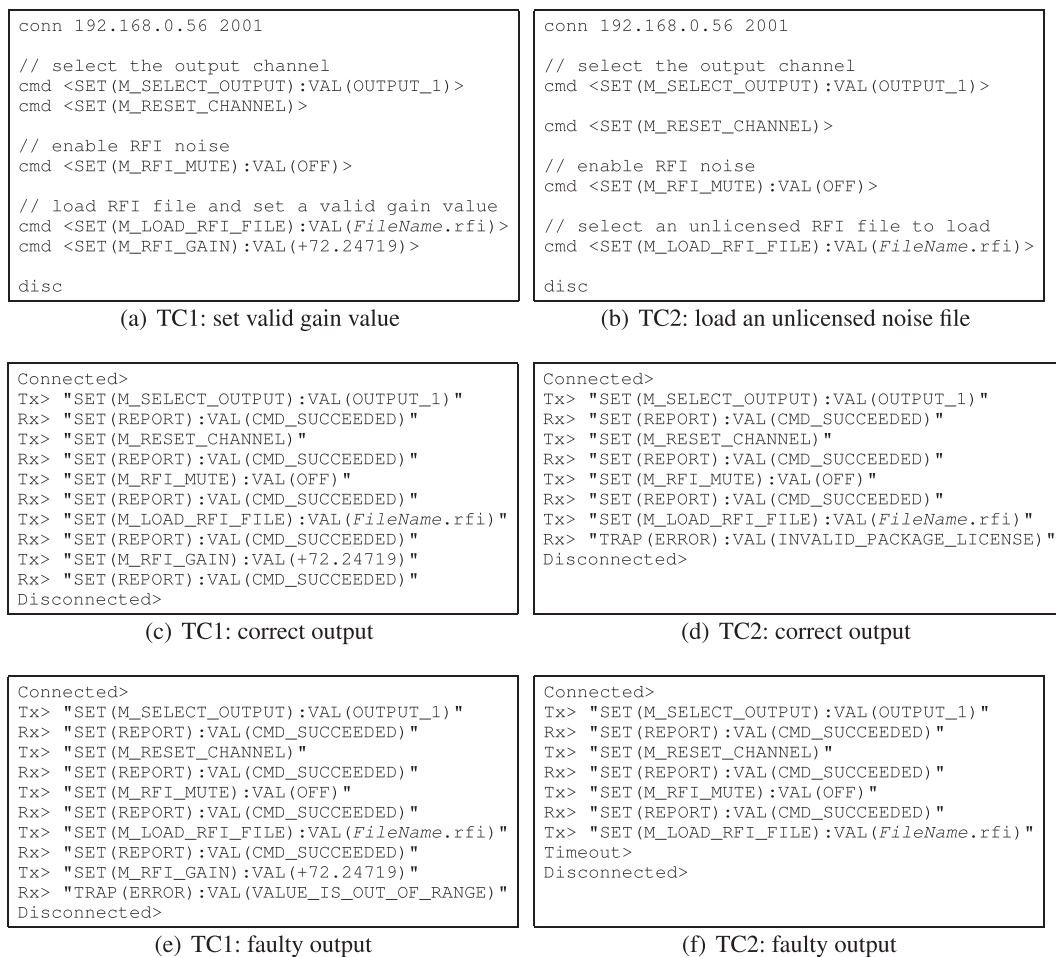


Figure 1. Two example test cases. Each column shows a test case, a correct output log and a faulty output log for that test case.

specifically targeting the uncovered code, the test cases generated for each functional testing unit were reevaluated to modify the definitions of categories and choices that had led to the functionality being under-tested. The test generation process was stopped when all reachable code was covered. Reachable code was determined manually by examining uncovered code; the strongest available code coverage criterion, decision, was used in the analysis. Examples of unreachable code in the NoiseGen system are functions in internal company libraries that are not used for this system.

The test cases were generated by the first author of this paper who, at the time of the case study, was an employee of the company that developed the NoiseGen system and, therefore, had the domain knowledge to aid in the generation of test cases. The user manuals of the NoiseGen system were also used to guide the generation of an initial set of test cases. Each test case is a script consisting of a sequence of NoiseGen system commands. Figure 1(a) and 1(b) show examples of test cases. The script of Figure 1(a) selects a type of noise to be outputted (an RFI file) and also sets a parameter value related to the noise output (the gain). The script of Figure 1(b) attempts to load a type of noise that is not licensed for use on the NoiseGen system. A test driver was written in C++ to parse each script and send the commands to the NoiseGen system via Transmission Control Protocol/Internet Protocol.

For each version, new test cases were generated following the same procedure to cover new or modified functionalities. The test cases were ordered in the initial test suite such that test cases generated for each functional testing unit (i.e. command) were grouped together. The commands are ordered following the order in which they appear in the user manual. For each subsequent system version, additional test cases required to test any new functionality were added to the end of the existing test suite.

An output log file is saved for each test case execution. For each newly created test case, the tester verified its corresponding output log. After a regression test suite was executed, a text diff tool was used to compare the regression test output logs against their expected values. That is, the outputs were compared with the saved output logs of a successful execution of the same test cases on the previous version of the software. The tester acted as the oracle, manually inspecting any differences found in the output logs and deciding if differences were caused by obsolete or failing test cases. A test case was considered obsolete if it tested functionality that was removed or modified in the new version of the system (e.g. a command was removed). Figure 1(c) and 1(d) demonstrate the correct output logs of the examples of Figure 1(a) and 1(b), while Figure 1(e) and 1(f) demonstrate faulty outputs for the same examples. Lines starting with Tx> denote executed commands while lines starting with Rx> denote the response from the system. In Figure 1(e), a value that was previously accepted is now rejected as out-of-range while the range specifications did not change. In the output in Figure 1(f), instead of rejecting a noise that is not licensed to use NoiseGen, the system times out.

Table II provides information about the test suites used for each version of the system. The column *Total Size* indicates the number of test cases used to test a specific version. This includes all test cases from the previous version that did not become obsolete together with the new test cases generated specifically for that version. For example, for version 2, *Total Size* is the non-obsolete test cases from version 1 ($541 - 36 = 505$) together with the new test cases generated for version 2 ($505 + 228 = 733$).

The column *Regression Cases* indicates the number of test cases used in the regression test suite for each version. The regression test suite contains all the test cases that executed successfully

Table II. Test suite details for each version: the number of new test cases generated for each version, failed and obsolete test cases and the size of the regression test suite.

Version	Total Size	New	Obsolete	Failed	Regression Cases
1	541	541	–	158	–
2	733	228	36	134	347
3	772	39	0	18	599
4	875	103	0	25	754
5	876	2	1	34	849

on the previous version and did not become obsolete on the current version. For example, for version 2 this will be 347 ($541 - 158 - 36$). Failed tests are excluded for practical reasons: because the test failed on the previous version, no oracle is available for subsequent versions of the system. The execution time of each test case is, on average, two minutes. Executing the regression test suite for NoiseGen takes between 11.6 (for version 2) and 28.3 hours (for version 5). In practice, during development, manual tests were run on the NoiseGen system on a daily basis. The lengthy execution time of the regression test suites emphasizes the need for regression testing methodologies.

4.3. Identifying faults

This study uses real regression faults to evaluate the studied techniques. Producing seeded faults can be automated, and a large number of faults can be produced to generate more experimental data, however, at the expense of less external validity or realism [22]. Identifying real faults, on the other hand, is time-consuming and the number of faults found might not be sufficient to generate statistically significant data when regression testing techniques are applied to the system.

Unfortunately, the NoiseGen system developers did not maintain a bug repository or document changes to the system that were made to correct faults. Therefore, faults had to be located manually using the system's expected behaviour, as described by the specifications in the user manual, and by performing an analysis of code changes in subsequent versions.

The faults were identified by one of the authors who, at the time of the case study, was an employee of the company that developed the NoiseGen system, as this task required not only significant effort on past versions of the system but domain and system expertise as well. While executing each test suite, each test case failure was attributed to a fault. In certain cases, multiple test cases failed because of the same software fault. It was then determined which faults were regression faults, as opposed to faults in new or modified functionality. A fault is considered to be a regression fault when, although a particular behaviour is not expected to change from the previous version, at least one test case execution shows this to be the case.

Table III reports the number of regression faults that were identified in each version and the number of fault detecting test cases. More faults might exist in the code than those reported in the table. However, only faults that were detected by the regression test suite are reported.

4.4. Effectiveness measures

Rothermel *et al.* [17] presented a metric that has since become the standard to measure the efficiency of prioritization techniques. This metric measures how quickly a test suite detects faults by calculating the weighted average of the percentage of faults detected (APFD) over the life of a test suite. The APFD value ranges from 0 to 100. The higher the APFD number is, the faster the rate of fault detection.

Elbaum *et al.* [15] presented the equation to calculate the APFD measure. For a test suite T with n test cases and m number of faults F , the APFD metric would be calculated as follows:

$$APFD = \left[1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n} \right] \times 100,$$

Table III. The number of regression faults detected for each version together with the number of test cases that detected them.

Version	Faults	Test Cases
2	30	113
3	5	10
4	1	10
5	1	12
Total	37	145

where TF_i is the position of the first test case that reveals fault i . Note that this metric treats all faults as having uniform severity. Elbaum *et al.* [23] also presented a version of this metric, $APFD_C$, that incorporates varying test case and fault severity costs. Because the severity of faults cannot be determined without introducing bias to the experiment, the standard APFD measure is used to evaluate the four prioritization techniques used in this study.

To evaluate test case selection, test suite minimization and the hybrid approach, the metrics proposed by Rothermel *et al.* [24] are used. Using these approaches allows for potentially dramatic reductions in test execution times. The savings in time are mostly dependent on the number of tests that can be removed from the test suite. Thus, a measure representing the percentage reduction in test suite size can represent the savings:

$$\text{Percentage Reduction in Test Suite Size} = \frac{|T| - |T_{post}|}{|T|} \times 100,$$

where $|T|$ is the size of the original test suite while $|T_{post}|$ is the size of the test suite after applying test case selection, test suite minimization or the hybrid approach. This is a simple approach that makes several assumptions, including that all test cases are equal in cost (e.g. requiring equal human or equipment times).

To investigate the effect of applying selection and minimization techniques on the effectiveness of the resulting test suite, the faults that are missed as a result of discarding test cases are examined. A measure representing the percentage reduction in fault detection effectiveness is given by the following:

$$\text{Percentage Reduction in Fault Detection Effectiveness} = \frac{|F| - |F_{post}|}{|F|} \times 100,$$

where $|F|$ is the number of faults revealed by the original test suite T over a faulty version of the program P and $|F_{post}|$ denotes the number of faults revealed by T_{post} over the same version of P . This measure assumes that the costs associated with missed faults are equal.

4.5. Data collection

For each version of the application, the relevant test suite was executed to obtain coverage information. The xSuds tool was then used on each version (starting from version 2) to apply the 35 different techniques (four prioritization, one selection, one minimization and one hybrid strategy with five coverage levels), on the regression test suite described in Section 4.2 and Table II. For each version of the system, the tool uses the coverage information of its immediate predecessor version.

The system was initialized (by restarting) after each test case was executed to ensure that test cases behaved consistently over multiple runs and eliminate the effects of interdependencies between test cases. Test cases were executed automatically and the outputs were saved for later analysis.

Because there is a random component in the studied techniques, each experiment was repeated 100 times for each version. When using the xSuds tool, it was observed that the results of each technique were sensitive to the initial order of test cases provided to the tool with the exception of the test selection technique, which is deterministic. This is because xSuds uses the initial order of the test cases to resolve ties when multiple test cases have the same coverage; there is no random selection. Therefore, to account for the fact that the original ordering of the regression test suite is arbitrary and is a factor of randomness, 100 different initial test case orders were randomly sampled to provide as input to the tool. The optimal order of test cases was also manually produced to provide the theoretically maximum possible APFD score when investigating prioritization techniques. Each of the resulting test suites was then executed on the relevant system version and the fault data collected.

To compare the different prioritization techniques, the APFD measure described in Section 4.4 is used for each technique and each version of the system. Additionally, to allow for a comparison of the techniques to be made across all versions, an average APFD score is calculated for all the versions combined for each technique. This is a weighted average that gives more weight to

those versions with more faults. Given that 37 faults were detected across the four system versions considered, then

$$APFD_{All} = \sum_{i=2..5} \frac{m_{vi}}{37} \times APFD_{vi},$$

where i is the version number, m_{vi} is the number of faults found in version i , and $APFD_{vi}$ is the APFD score for version i . A weighted average is used to perform an overall comparison using all the data obtained from all versions so that less weight is assigned to APFD values for versions based on fewer faults.

To compare the different test selection, test suite minimization and the hybrid techniques, the Percentage Reduction in Test Suite Size and Percentage Reduction in Fault Detection Effectiveness measures described in Section 4.4 are used. To be able to compare the techniques across all versions, the average for both measures for all versions combined for each technique and each coverage criterion is also calculated.

4.6. Analysis

To analyze and draw conclusions from the collected data, the recommendations of Arcuri and Briand [25] regarding data analysis are followed. Wilcoxon's paired two-sided signed rank test at the 95% confidence level is used to determine the statistical significance of observed differences in APFD scores, percentage reduction in test suite size and percentage reduction in fault detection effectiveness across techniques. Wilcoxon's test is used because it is non-parametric, and assumptions should not be made on the distribution of measures. The test is paired because the same initial random order of test cases is used for each of the different techniques on each of the 100 trials.

Vargha-Delaney effect size measure $\hat{A}_{12}$ is also calculated because it is non-parametric. When comparing two techniques, $\hat{A}_{12}$ measures the probability that one technique would perform better than the other technique. A value of 0.5 would mean that the two techniques have equal probability of performing better than the other.

5. RESULTS

This section reports and discusses the results of applying the different prioritization, selection and minimization techniques as well as the hybrid technique on the four versions of the system.

5.1. Test case prioritization

Figure 2 summarizes and depicts, using standard boxplots, the overall results of applying the different prioritization techniques by showing the variations in the weighted APFD scores across all techniques.

5.1.1. The effect of coverage criteria. Figure 2 shows that although the granularity level of coverage affects results in all approaches, the effect is much more pronounced in techniques that are based on additional coverage. When examining the boxplots for those two additional coverage techniques, one observes that the performance of coverage criteria, as indicated by APFD scores, can be classified into two groups where one group clearly outperforms the other. In both additional coverage and additional coverage of modified code techniques, the finer grained coverage criteria (block, basic block and decision) perform similarly overall and significantly outperform the coarser grained coverage criteria (function entry and function return).

Table IV provides the detailed results for the additional coverage technique and for all coverage criteria. The table reports the mean and median APFD scores over 100 runs for each of the system's versions individually for each of the coverage criteria. The results confirm the previous observation and show that for versions 2, 4 and 5, the mean and median APFD scores of the finer grained coverage criteria are significantly higher than those of the coarser grained coverage criteria.

The only exception is version 3, where the APFD scores for function entry and function return are slightly higher than block, basic block and decision coverage criteria. The difference is not

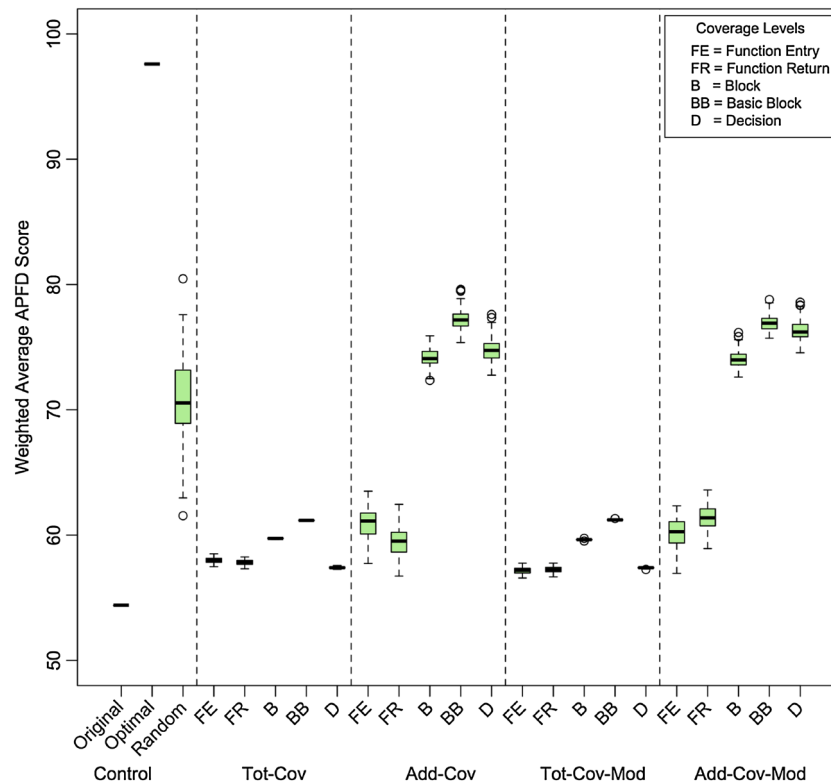


Figure 2. Variation in the weighted average of the percentage of faults detected scores across all the versions for each of the prioritization techniques used in the study. Tot-Cov, total coverage; Add-Cov, additional coverage; Tot-Cov-Mod, total coverage of modified code; Add-Cov-Mod, additional coverage of modified code; APFD, average of the percentage of faults detected.

Table IV. Mean and median average of the percentage of faults detected scores for the additional coverage technique for all coverage criteria over 100 runs on each version of the system.

Version	Function Entry		Function Return		Block		Basic Block		Decision	
	mean	median	mean	median	mean	median	mean	median	mean	median
2	57.7	57.8	56.7	56.6	74.4	74.3	78.0	78.0	74.3	74.1
3	72.1	71.9	72.7	72.7	67.4	67.6	67.0	67.0	69.9	71.1
4	82.9	84.2	64.8	62.9	91.2	91.3	90.1	89.3	93.3	93.4
5	79.7	77.6	69.0	70.1	84.1	84.2	89.8	89.8	96.7	96.8
All	60.9	61.1	59.4	59.5	74.2	74.1	77.2	77.2	74.8	74.7

statistically significant and, therefore, might be due to chance. One also observes that the performance of the studied prioritization techniques is lowest overall for this version. This might be because of the types of faults found in version 3 that might be harder to detect and possibly not dependent on coverage. Recall from Table III that version 3 has five faults that are detected by 10 test cases while all other versions have a much higher ratio between the number of faults and the number of fault detecting test cases. This could suggest a relationship between the number of fault detecting test cases and the overall success of coverage-based test case prioritization. However, more empirical studies are needed to investigate this relationship.

To test the statistical significance of the observed results, in this section, block coverage is selected to represent the finer grained coverage criteria and function entry coverage to represent coarser grained coverage criteria. Table V reports the results of statistical testing for the block coverage criterion for the two additional coverage approaches to function entry coverage, random order and

Table V. Statistical tests results from comparing the weighted average of the percentage of faults detected scores for the two additional coverage techniques for block coverage to the same techniques for function entry coverage, random ordering and optimal order.

Block versus	Additional Coverage		Additional Coverage of Modified Code	
	<i>p</i> -value	$\hat{A}_{12}$	<i>p</i> -value	$\hat{A}_{12}$
Function entry	<.001	1.00	<.001	1.00
Random	<.001	0.83	<.001	0.82
Optimal	<.001	0.00	<.001	0.00

Table VI. Mean and median average of the percentage of faults detected scores for all prioritization techniques for block coverage over 100 runs on each version of the system.

Version	Total Coverage		Additional Coverage		Total Coverage of Modified Code		Additional Coverage of Modified Code	
	mean	median	mean	median	mean	median	mean	median
2	57.9	57.9	74.4	74.3	58.3	58.3	75.0	74.8
3	70.6	70.6	67.4	67.6	65.0	65.0	60.4	60.3
4	55.4	55.4	91.2	91.3	61.1	61.1	93.0	93.2
5	64.5	64.5	84.1	84.2	72.4	72.4	96.2	99.2
All	59.7	59.7	74.2	74.1	59.6	59.6	74.1	74.0

the optimal order. The first row in Table V reports the *p*-values obtained by performing Wilcoxon's paired two-sided signed rank test and the Vargha-Delaney effect size measure ($\hat{A}_{12}$) from comparing the block coverage criterion to the function entry coverage criterion for both additional coverage approaches. The statistical tests are calculated for the weighted APFD score means across all versions. The results show that for both approaches, the block coverage criterion is statistically significantly better at achieving higher APFD scores and performs better 100% of the time (effect size is equal to one) than the function entry coverage criterion. Nested statistical testing (per version) confirms similar results except for versions 4 and 5 where the two approaches are not statistically significantly better than the random approach.

5.1.2. The effect of the prioritization approach. Figure 2 shows that the two approaches based on additional coverage using finer grained coverage criteria yield the highest APFD scores of all applied prioritization approaches.

Table VI provides the detailed mean and median APFD scores for each version of the system for the block coverage criterion (one of the fine grained criteria) for all approaches. The results show that for versions 2, 4 and 5, the additional coverage approaches perform better than the total coverage approaches.

For version 3, the total coverage approach obtains a higher APFD score than the additional coverage approaches for block coverage. However, looking back at Table IV, additional coverage techniques still yield the highest APFD scores for this version when comparing all results, but for coarser grained coverage criteria instead of the block coverage criterion (72.1 and 72.7 compared with 70.6 for total coverage using block coverage).

Table VII reports the results of statistical testing (*p*-values) and effect size. The results show that the additional coverage based approaches are statistically significantly better than the total coverage based approaches with an effect size of one. It also shows that the total coverage approach is statistically significantly better than the total coverage of modified code approach. Nested statistical testing (per version) shows that the results are statistically significantly different for each version.

The difference between the performance of the two additional coverage based approaches is not statistically significant. However, one might conclude that the additional coverage of modified code

Table VII. Statistical testing results of pair-wise comparison of mean weighted average of the percentage of faults detected scores for 100 runs over all versions for all four prioritization techniques using block level coverage.

	Total Coverage		Additional Coverage		Total Coverage of Modified Code		Additional Coverage of Modified Code	
	<i>p</i> -value	$\hat{A}_{12}$	<i>p</i> -value	$\hat{A}_{12}$	<i>p</i> -value	$\hat{A}_{12}$	<i>p</i> -value	$\hat{A}_{12}$
Total coverage			<.001	0.00	<.001	0.98	<.001	0.00
Additional coverage	<.001	1.00			<.001	1.00	.107	0.56
Total coverage of modified code	<.001	0.02	<.001	0.00			<.001	0.00
Additional coverage of modified code	<.001	1.00	.107	0.44	<.001	1.00		

Table VIII. Mean and median average of the percentage of faults detected scores for the additional coverage technique using block coverage and random ordering over 100 runs together with the scores for the optimal and original order for each version of the system.

Version	Additional Coverage		Random		Optimal	Original
	mean	median	mean	median		
2	74.4	74.3	70.9	71.4	97.1	56.9
3	67.4	67.6	61.8	60.7	99.7	23.4
4	91.2	91.3	91.3	94	99.9	92.2
5	84.1	84.2	92.2	93.9	99.9	95.8
All	74.2	74.1	70.8	70.6	97.6	54.4

approach is more expensive because it requires the two versions of the system to be analyzed to identify the modified code.

Answer to RQ1: How does the choice of test case prioritization technique and granularity of coverage criteria used in the analysis affect the rate of fault detection?

Prioritization techniques based on finer grained coverage criteria fare better than those based on coarser grained coverage criteria. The results also show that this effect can be observed much more strongly in prioritization techniques that are based on additional coverage, which also turn out to be the best techniques. This difference is statistically significant at a 95% confidence level. Techniques based on additional coverage perform statistically significantly better than techniques based on total coverage. Techniques based on coverage of modified code do not perform significantly better than those that do not use modification information. Therefore, the additional coverage technique is recommended, given the fact that the additional coverage of modified code technique is more expensive as it requires additional source code analysis to obtain modification information.

5.1.3. The effectiveness of prioritization techniques. Table VIII compares the mean and median APFD scores over 100 runs of the additional coverage approach using block coverage (one of the best prioritization results) to the APFD scores for random ordering. It also reports the APFD scores of the test suites in their original order for each version of the system and the APFD scores of the optimal orders.

The results show that additional coverage performs better than random ordering and the original order. The difference between the mean APFD scores for additional coverage using block coverage and random order is 3.4 (74.2 – 70.8). In addition, as can be seen in Figure 2, the additional coverage technique performs more consistently than random (i.e. it has a much lower standard deviation). The weighted average APFD scores for random ordering range from 61.5 to 80.5 while those for additional coverage using block coverage range from 73.3 to 75.9. In other words, with random orders you may get lucky, but a coverage approach is much more predictable. The results of statis-

tical testing in Table V show that the additional coverage and additional coverage of modified code techniques perform statistically significantly better than random ordering, with a standardized effect size of 0.83 and 0.82, respectively.

Tables VIII and V also show that the difference in APFD scores between the prioritization techniques and the optimal order are statistically significant. The large differences in all versions suggest that there is still significant room for improvement with standard prioritization techniques, which may not be achievable through common coverage-based approaches.

The APFD scores for both the additional coverage technique and random ordering for versions 4 and 5 are the highest of all versions and closest to the optimal order scores. This might indicate that the faults of versions 4 and 5 are easier to detect than the faults of the other versions, thus contributing to these high APFD scores. The nature of the code changes made for these versions might also be a factor in reaching these more favourable values. Recall from Table III that versions 4 and 5 contain only one regression fault each, and these specific results should not be given much weight individually. The faults in these two versions were related to the system accepting invalid parameters or disconnecting instead of reporting an invalid input file. Recall that nested statistical testing revealed that for versions 4 and 5, block coverage using the additional coverage and additional coverage of modified code techniques were not found to perform statistically significantly better than random. This might be because of the fact that the faults of versions 4 and 5 have a high number of fault detecting test cases (10 and 12, respectively), making a random approach more likely to perform better than in a version where the number of fault revealing test cases for each fault are lower. In versions 2 and 3, where the techniques were found to perform statistically significantly better, there were on average 3.8 and two fault detecting test cases, respectively.

Answer to RQ2: Can coverage-based test case prioritization improve the rate of fault detection of a regression test suite?

Using prioritization techniques based on additional coverage does, in fact, significantly improve the fault detection rate of a regression test suite over random ordering and the original test suite. However, the APFD scores of these prioritization techniques are significantly lower than optimal. This suggests that better prioritization techniques still need to be investigated, probably based on entirely different principles than coverage and code modification analysis.

5.2. Test case selection

This section discusses the results of applying the coverage-based test case selection technique to the four versions of the system using the five coverage criteria. Section 5.2.1 discusses the effect on execution cost, while section 5.2.2 examines the effect on fault detection.

5.2.1. Execution cost. As mentioned previously, test suite size can be used as a surrogate for the cost of executing a test suite and, therefore, the reduction in the test suite size after applying the test selection technique can indicate how useful the approach is in practice. Table IX reports the percentage reduction in test suite size when applying test case selection to the regression test suites of the four versions of the system using the five coverage criteria. The results show that no reduction is observed in most cases, and the maximum reduction that was achieved is less than 2%. The results also show that the coverage criterion used has little impact on the results.

Table IX. Percentage reduction in test suite size after applying test case selection on the four versions of the case study system using the five coverage criteria.

Version	Function Entry	Function Return	Block	Basic Block	Decision
2	0.0	0.0	0.0	0.0	0.0
3	1.2	0.2	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0
5	1.6	1.8	1.6	1.6	1.6
All	0.8	0.6	0.5	0.5	0.5

The reason test selection did not offer significant reductions in execution cost in the case study system could be attributed to the location of the changes to the system. If the modifications to the system affect a core component, it is likely that the majority of test cases would cover the changed code and, therefore, would not be discarded. This explanation is also supported by the results and investigations reported in related work (which will be discussed in more detail in Section 7). These results might indicate that the decision to use test case selection can be based on the changes made to the system and the nature of the release (minor versus major). Minor releases where the changes to the system are limited might benefit more from applying a test selection technique.

5.2.2. Fault detection. Table X reports the percentage reduction in fault detection effectiveness after applying test case selection using the five coverage criteria. It is not surprising that no reduction in fault detection is observed since applying the technique only led to a very small reduction in test suite size as reported in Section 5.2.1. The level of coverage criteria has no impact on fault detection effectiveness.

Answer to RQ3: Can test case selection techniques reduce the cost of executing a test suite and what is their effect on the fault detection effectiveness of a test suite?

Coverage-based test case selection techniques provided no or very small (<2%) reduction in execution cost and no reduction in fault detection effectiveness using all five coverage criteria and in all four versions of the case study system.

5.3. Test suite minimization

This section discusses the results of applying test suite minimization to the four versions of the case study system using the five coverage criteria. Section 5.3.1 discusses execution cost, while Section 5.3.2 discusses the reduction in fault detection capabilities. Statistical testing for the results of the test suite minimization experiments was not performed because it is clear there is a large difference between the finer and coarser grained coverage criteria, no observable difference within each group of coverage criteria, and very little statistical variation within each criteria.

5.3.1. Execution cost. Table XI reports the mean and median percentage reduction in test suite size over 100 runs for each of the four versions of the system for each of the five coverage criteria. The results show that test suite minimization can significantly reduce the size of a test suite. The results

Table X. Percentage reduction in fault detection effectiveness after applying test case selection on the four versions of the case study system using the five coverage criteria.

Version	Function Entry	Function Return	Block	Basic Block	Decision
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0
5	0.0	0.0	0.0	0.0	0.0
All	0.0	0.0	0.0	0.0	0.0

Table XI. Percentage reduction in test suite size after applying test suite minimization on the four versions of the case study system using the five coverage criteria.

Version	Function Entry		Function Return		Block		Basic Block		Decision	
	mean	median	mean	median	mean	median	mean	median	mean	median
2	96.5	96.5	95.4	95.4	70.3	70.3	70.3	70.3	72.0	72.0
3	97.3	97.3	91.7	91.7	82.1	82.1	82.1	82.1	81.8	81.8
4	97.6	97.6	93.1	93.1	81.0	81.0	81.0	81.0	79.3	79.3
5	97.5	97.5	93.4	93.4	80.1	80.1	80.1	80.1	78.2	78.2
All	97.4	97.4	93.2	93.2	79.5	79.5	79.5	79.5	78.5	78.5

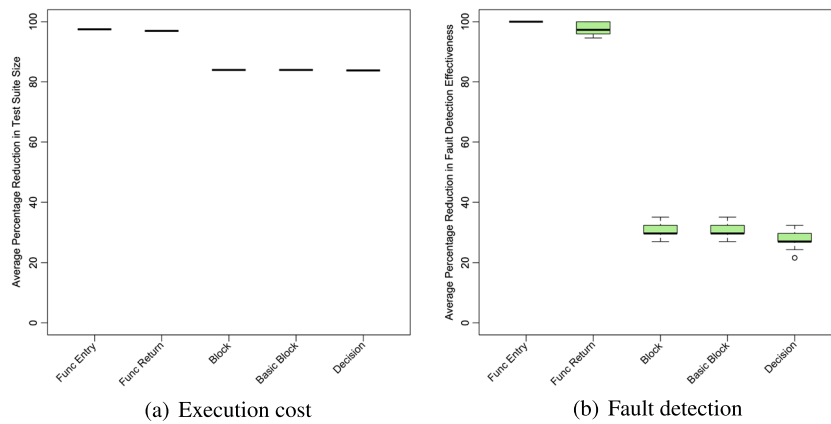


Figure 3. Variations over 100 runs in the average over all versions of the system of the percentage reduction in test suite size and the percentage reduction in fault detection effectiveness over 100 runs when applying test suite minimization for the five coverage criteria. Func Entry, function entry; Func Return, function return.

Table XII. Percentage reduction in fault detection effectiveness after applying test suite minimization on the four versions of the case study system using the five coverage criteria.

Version	Function Entry		Function Return		Block		Basic Block		Decision	
	mean	median	mean	median	mean	median	mean	median	mean	median
2	100.0	100.0	98.3	96.7	22.7	23.3	22.7	23.3	20.3	20.0
3	100.0	100.0	100.0	100.0	77.8	80.0	77.8	80.0	65.8	60.0
4	100.0	100.0	100.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0
5	100.0	100.0	100.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0
All	100.0	100.0	98.6	97.3	28.9	29.7	28.9	29.7	25.4	24.3

also show that the coarser grained coverage criteria yield a larger reduction in test suite size (93.2–97.4%) than the finer grained criteria (78.5–79.5%). Figure 3(a) illustrates the variations in average percentage reduction in test suite size over all versions of the system for the different coverage criteria. One can observe that there is little variation in the reduction in test suite size. This could be caused by the coverage of the test cases, that is, if the coverage of individual test cases is similar, the results of minimizing with regards to a coverage criterion may result in a similar number of test cases.

5.3.2. Fault detection. Table XII reports the mean and median reduction in fault detection effectiveness over 100 runs for each of the four versions of the system and each of the five coverage criteria. Figure 3(b) summarizes the results across all versions for each of the five coverage criteria. Test suites minimized using the coarser grained coverage criteria (function entry and function return) result in a complete failure to detect faults in most cases. The finer grained coverage criteria (block, basic block and decision) perform differently for different versions but overall perform significantly better than the coarser grained coverage criteria (25.4–28.9% reduction compared with 98.6–100%).

Recall that the finer grained coverage criteria yield a significant reduction in test suite execution cost. Therefore, considering the reported reduction in fault detection effectiveness, the technique might be useful in practice in non-critical systems and when the time allocated for testing is limited. Although the test suite minimization technique, by definition, maintains the same level of coverage as the original test suite, the reduced test suites do not have the same fault detection capabilities as the original test suite. These results provide an additional data point that supports previous studies [26–29] that found that coverage is not the only factor that affects fault detection.

Table XIII. Percentage reduction in test suite size after applying the hybrid technique on the four versions of the case study system using the five coverage criteria.

Version	Function Entry		Function Return		Block		Basic Block		Decision	
	mean	median	mean	median	mean	median	mean	median	mean	median
2	96.5	96.5	96.8	96.8	72.9	72.9	72.9	72.9	76.1	76.1
3	97.3	97.3	96.2	96.2	89.1	89.1	89.1	89.1	88.8	88.8
4	97.7	97.7	97.5	97.5	83.0	83.0	83.0	83.0	83.3	83.3
5	97.8	97.8	97.1	97.1	85.6	85.6	85.6	85.6	83.9	83.9
All	97.5	97.5	96.9	96.9	84.0	84.0	84.0	84.0	83.8	83.8

Answer to RQ4: Can test suite minimization techniques reduce the cost of executing a test suite and what is their effect on the fault detection effectiveness of a test suite?

Coverage-based test suite minimization can produce significant reductions in test suite size, resulting in significant savings in test execution cost. The coarser grained coverage criteria provide a higher reduction in test suite size compared with the finer grained coverage criteria. Test suite minimization using coarser grained coverage criteria causes the test suite to be ineffective at detecting faults. However, when using finer grained coverage criteria, test suite minimization causes an average reduction in fault detection effectiveness that is less than 29%. This makes the approach potentially useful in practice, when testing time is limited and the system is not critical.

5.4. Hybrid technique

This section discusses the results of a combined approach that first applies test case selection and then applies test suite minimization based on coverage of the modified code. The experiment is repeated for each of the four versions of the system and the five coverage criteria. Section 5.4.1 discusses execution cost, while Section 5.4.2 discusses the reduction in fault detection capabilities. Statistical testing was not performed because, similar to test suite minimization, there is a large difference between the finer and coarser grained coverage criteria, no observable difference within each group of coverage criteria, and very little statistical variation within each criteria.

5.4.1. Execution cost. Table XIII reports the mean and median percentage reduction in test suite size over 100 runs after applying the hybrid technique on each of the four versions and each of the five coverage criteria. The results show that the hybrid technique provides a higher reduction in test suite size than the reduction achieved by both test case selection and test suite minimization. This result is expected because test suite selection does not provide significant reduction in test suite size, and the hybrid approach only considers modified code when minimizing, compared with test suite minimization that considers all code. However, the additional reduction compared with minimization might not be considered significant enough to justify the extra effort. Similar to test suite minimization, the coarser grained coverage criteria perform better than the finer grained criteria.

Figure 4(a) summarizes the results over all versions for each of the five coverage criteria. One can observe that, similar to minimization, there is little variation in the reduction in test suite size. This is expected because the hybrid technique uses the same test suites and applies a similar test reduction technique as part of its algorithm.

5.4.2. Fault detection. Table XIV reports the mean and median reduction in fault detection effectiveness over 100 runs after applying the hybrid technique for each of the four versions of the case study system and each of the five coverage criteria. Figure 4(b) depicts the variation in the percentage reduction of fault detection effectiveness over all versions of the system for each of the five coverage criteria.

The results are similar to the results of test suite minimization, however, the reduction for the finer grained coverage criteria is higher than that reported for test suite minimization. Overall, the hybrid technique provides a slightly higher reduction in test suite size but causes a greater loss in

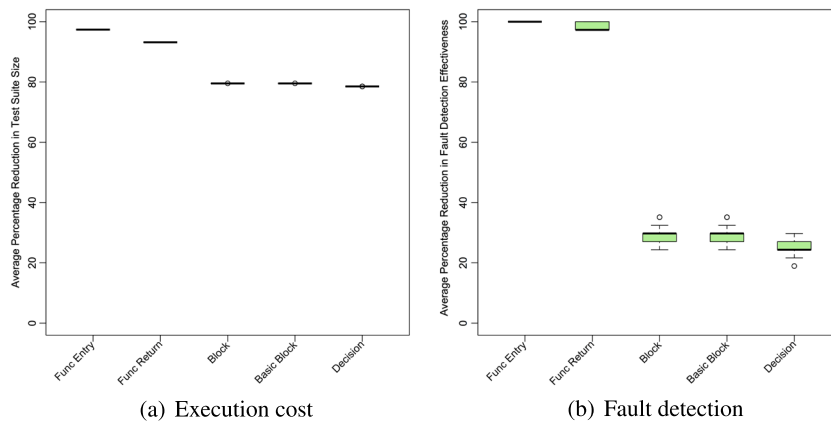


Figure 4. Variations over 100 runs in the average over all versions of the system of the percentage reduction in test suite size, and the percentage reduction in fault detection effectiveness when applying the hybrid technique for the five coverage criteria. Func Entry, function entry; Func Return, function return.

Table XIV. Percentage reduction in fault detection effectiveness after applying the hybrid technique on the four versions of the case study system using the five coverage criteria.

Version	Function Entry		Function Return		Block		Basic Block		Decision	
	mean	median	mean	median	mean	median	mean	median	mean	median
2	100.0	100.0	98.4	100.0	22.7	23.3	22.7	23.3	20.4	20.0
3	100.0	100.0	91.2	100.0	85.8	80.0	85.8	80.0	82.0	80.0
4	100.0	100.0	100.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0
5	100.0	100.0	100.0	100.0	0.0	0.0	0.0	0.0	0.0	0.0
All	100.0	100.0	97.5	97.3	30.0	29.7	30.0	29.7	27.6	27.0

fault detection effectiveness. These results, combined, indicate that a hybrid technique might not be useful in practice, considering the additional effort required compared with just applying test suite minimization.

Answer to RQ5: How does a hybrid approach that combines test case selection and test suite minimization perform compared with the individual approaches in terms of cost reduction and fault detection effectiveness?

The hybrid technique outperforms both test case selection and test suite minimization in terms of test suite size reduction. However, the extra reduction compared to test suite minimization is not significant enough for the technique to be beneficial. The hybrid technique also causes a higher reduction in fault detection effectiveness than both test case selection and test suite minimization.

6. THREATS TO VALIDITY

This section briefly discusses the threats to validity that are present in this study following the standard classification of threats [30].

Internal Threats: The internal threats that can affect the validity of results are related to the execution of the experiments and the detection of faults. The results might have been affected by the selected tool; however, the same tool was used to perform all techniques, and 100 orderings were used for all non-deterministic techniques. The output logs of failed test cases could not be used by the oracle of the successor version; therefore the failed test cases were excluded from the regression test suite of the successor version and this might have resulted in decreased fault-detection ability.

External Threats: The experiments were conducted on a single industrial system; this limits the extent to which the results can be generalized. The design of the system and the nature of the changes

made on each version might affect the performance of the different techniques. The test cases were generated by one of the authors. To reduce this risk, the test cases were created following a systematic black-box strategy to minimize bias and the test cases were created before the experiments were designed. All of the test cases were generated and executed after the release of all the software versions used in the study, however the faults used were real and in some cases the faults detected by the test suite were fixed in a subsequent release. The NoiseGen system has a core component and an API that are affected by most changes to functionality. This design might cause prioritization approaches that use modification information to add no value, as was observed in the results. This design might also cause the test selection technique to provide limited reduction in test suite size and, therefore, no reduction in fault finding effectiveness (as observed in Tables IX and X) because the majority of test cases are likely to traverse the changes in the core component and API. However, such a design might be common in industrial systems and more investigations are needed to confirm this observation.

Construct Threats: This case study assumes all test cases are equal in cost. This is generally true in that each test script takes, on average, two minutes to execute. There are slight variations in execution time (measurable in seconds) that could be taken into account to produce more accurate results. This experiment also assumes that faults are equal in severity. It is possible that some faults might be more severe and may, therefore, have a higher associated cost.

7. RELATED WORK

In this section, the findings of this case study are compared to existing industrial and non-industrial studies that investigate coverage-based regression testing techniques.

Yoo and Harman's [2] survey of regression testing was used to guide the search; the search also targetted case studies that were published since the survey was completed. In deciding about whether to include industrial studies, the selection criteria was defined as follows: the study has to be applied to an industrial system and uses at least one of the techniques used in this study. The study also has to report the values of the APFD scores obtained for prioritization and reduction in test suite size and/or reduction in fault detection effectiveness for selection, minimization or the hybrid technique. Based on these selection criteria, some relevant studies that did not satisfy the criteria had to be excluded. For example, Srivastava and Thiagarajan [31] do not report APFD scores in their study of prioritization.

7.1. Prioritization

Seven studies that applied prioritization techniques to industrial software [15, 31–36] were found. However, out of the seven studies, only two matched the selection criteria. In addition to those two studies, this case study's results were also compared to non-industrial empirical studies that use some of the same techniques.

Table XV summarizes all the studies that are used for the comparison. The table reports a description of the system(s) used in each study including the size, the number of versions used, the type and number of faults used, the number of test cases and all of the techniques used in the study that have also been used in this study together with the APFD scores for each technique.

The APFD scores reported in the table are the range of median scores across all systems/versions. The medians are reported as this is the only value that was reported for all techniques in all considered papers. The values are extracted from the boxplots of APFD scores and are, therefore, an approximation. Note that in this study's results, the mean and median APFD scores are similar, as can be seen in the results tables.

The first observation that can be made is that there are few studies of coverage-based prioritization techniques that use industrial systems and real regression faults (only two). One can also observe that in those two studies, only a limited combination of techniques and coverage criteria were applied. In fact, only one coverage criterion (function entry) has been studied.

The results of Elbaum *et al.* [15] contradict those of this study: total coverage approaches perform better than additional coverage approaches (77.3 and 75.6 compared with 66.3 and 69.6). In their

Table XV. Comparison to related work for test case prioritization.

Industrial Studies						
Study	System	Versions	Faults	Test Cases	Technique	Median APFD
This Study	NoiseGen (59–73 KLoC)	5	37 real regression faults	541–876	Original	54.4
					Random	70.6
					Optimal	97.6
					Tot Cov FE	58.0
					Add Cov FE	61.1
					Tot Cov Mod FE	57.2
					Add Cov Mod FE	60.3
					Tot Cov B	59.7
					Add Cov B	74.1
					Tot Cov D	57.4
					Add Cov D	74.7
Carlson <i>et al.</i> [32], 2011	Financial subsystem of Microsoft Dynamics Ax (650–705 KLoC)	3	reported by users	500–908	Tot Cov FE	46.6–50.8
Elbaum <i>et al.</i> [15], 2002	QTB: Embedded real-time subsystem that performs initialization tasks on level-5 RAID storage system (300+ KLoC)	6	17 real faults	135	Random	63.0
					Optimal	98.9
					Tot Cov FE	77.3
					Add Cov FE	66.3
					Tot Cov Mod FE	75.6
					Add Cov Mod FE	69.6
Non-Industrial Studies						
Elbaum <i>et al.</i> [11], 2003	Bash, Flex, Grep, Gzip (4–65 KLoC)	5–10	1–9 seeded faults per version	217–1168	Random	76.6–86.8
					Optimal	99.7–100
					Tot Cov FE	40.0–85.9
					Add Cov FE	87.3–98.1
Rothermel <i>et al.</i> [17], 2001	space (6 KLoC)	1	35 real faults	13585	Original	83.3
					Random	83.5
					Optimal	98.2
					Tot Cov D	92.3
					Add Cov D	92.5
					Random	82.2
					Optimal	98.9
					Tot Cov FE	75.1
					Add Cov FE	90.6
Elbaum <i>et al.</i> [16], 2004	Bash, Emp-server, Flex, Grep, Gzip, Make, Sed, Xearth (5–68 KLoC)	2–10	3–12 seeded faults per version	217–1985	Tot Cov Mod FE	79.2
					Add Cov Mod FE	86.8
					Original	30.7
					Random	65.5
					Optimal	98.5
Do <i>et al.</i> [14], 2004	Ant, XML-security, JMeter, JTopas (50–627 Classes)	4–9	5–21 seeded faults per system	78–877	Tot Cov B	54.4
					Add Cov B	94.6
					Tot Cov FE	51.6
					Add Cov FE	88.1
					Tot Cov Mod FE	61.8
					Add Cov Mod FE	77.0

Note: APFD, average of the percentage of faults detected; Tot Cov, total coverage; Add Cov, additional coverage; Tot Cov Mod, total coverage of modified code; Add Cov Mod, additional coverage of modified code; FE, function entry; B, block; D, decision.

Table XVI. Comparison to related work for test case selection, test suite minimization and the hybrid technique.

Industrial Studies									
Study	System	Versions	Faults	Test Cases	Technique	Size Reduction		Detection Reduction	
						Mean (%)	Median (%)	Mean (%)	Median (%)
This Study	NoiseGen (59–73 KLoC)	5	37 real regression faults	541–876	Selection FE	0.8	–	0.0	–
					Selection D	0.5	–	0.0	–
					Minimization FE	97.4	97.4	100.0	100.0
					Minimization B	79.5	79.5	28.9	29.7
					Minimization D	78.5	78.5	25.4	24.3
Rothermel <i>et al.</i> [4], 1998	Windows NT program (2 KLoC)	10	Not specified	388	Hybrid FE	97.5	97.5	100.0	100.0
					Hybrid D	83.8	83.8	27.6	27.0
					Selection D	66.0	–	0.0	–
					Selection D	47.2	–	0.0	–
					Minimization B	–	0.3	–	–
Hsu & Orso [37], 2009	LogicBox (570 KLoC), Eclipse (1.9 MLoC)	5	Not specified	393–3621					
Non-Industrial Studies									
Elbaum <i>et al.</i> [11], 2003	bash, flex, grep, gzip (4–65 KLoC)	5–10	1–9 seeded	217–1168	Minimization FE	–	92–98	–	10–60
					Hybrid FE	–	95–99	–	10–70
Graves <i>et al.</i> [6], 2001	space, player (6–49 KLoC)	1–6	33 real faults	1033–13 585	Selection D	–	80–95	–	0
					Hybrid D	–	98–99	–	0–90
Rothermel <i>et al.</i> [4], 1998	player (49 KLoC)	6	Not specified	1033	Selection D	95.0	–	0.0	–
					Selection FE	–	0–98	–	0.0
Harrold <i>et al.</i> [7], 2001	Siena, JEdit, JMeter, RegExp	5–11	Not specified	50–189	Minimization B	3–85	–	2–7	–
					Minimization D	22–97	–	8.9	–
Wong <i>et al.</i> [10], 1999	space (6 KLoC)	1	18 real faults	1000					
Rothermel <i>et al.</i> [38], 2002	space (6 KLoC)	1	35 real faults	13 585	Minimization B	3–85	–	2–7	–
					Minimization D	22–97	–	8.9	–

Note: FE, function entry; D, decision; B, block.

paper, this is attributed to the fact that fault exposing test cases in the system they used achieve a significantly higher total coverage of code than the average coverage of all other test cases. This might lead to these test cases being more likely to be ordered at the beginning of the test suite based on total coverage. These conditions might not apply in the case of this study, causing the results to differ.

This inconsistency in results suggests a need to investigate techniques that can be used in practice to analyze test cases and their coverage, so that one can select the prioritization technique that is more likely to perform better for a given system and test suite.

On the other hand, both studies show that using modification information does not improve the performance of the prioritization techniques.

When comparing the APFD scores of the industrial studies to the non-industrial studies, one can observe that APFD scores tend to be higher overall for non-industrial studies. This might be caused by the use of seeded faults and smaller, simpler software.

The random approach, as expected, performs inconsistently with APFD scores ranging from 63.0 to 86.8. In all studies but one [14], the improvement in median APFD scores for the best performing approach over the best median APFD scores for random orders ranges from 4.1 to 14.3. Based on such moderate improvements, one may question whether prioritization techniques are always cost-effective, as the analysis time to support prioritization could instead be used to run more test cases.

In all studies, one can observe that the optimal is close to 100 while the best performing prioritization techniques are still far from the theoretically possible optimum order. This suggests that it might be necessary to combine coverage approaches with other approaches that are based, for example, on models or historical data to improve performance.

7.2. Selection, minimization and the hybrid technique

Only two industrial case studies were found that report on test case selection, and only one study was found that reports on test suite minimization. This case study's results are also compared to non-industrial case studies that make use of similar techniques. The comparison of test case selection, test suite minimization and the hybrid technique is in the same section because the three approaches use the same effectiveness measures and, in many cases, the same study reports on more than one of the three approaches. Table XVI summarizes all the studies that are used for the comparison.

For test case selection, the other two industrial case studies reported significantly higher percentage reductions in test suite size (47.2% and 66%). For the non-industrial case studies, in some cases, reductions in test suite size that are much higher than those reported for industrial case studies were observed (95–98%). In one non-industrial case study, percentage reductions in test suite size were as small as the results reported in this study. Rothermel *et al.* [4] observed that fewer changes to the source code resulted in better performance of test selection techniques (i.e. higher reductions in test suite size) and also observed that when core components are changed, lower reduction in test suite size was reported. This observation is consistent with this study's findings, where test case selection yielded almost no reduction in test suite size and changes were made to core components for each successive software version. These results indicate that the effectiveness of test case selection techniques are correlated to the nature and size of the changes made to the system.

For test suite minimization, for both industrial and non-industrial case studies, the reduction in test suite size both match and disagree with this study's results. Naturally, the reduction in test suite size when using test suite minimization depends on the nature of the test suite (e.g. the percentage of redundant test cases) and therefore how it was formed (e.g. test strategy). Unfortunately, in the only minimization industrial case study [37], the reduction in fault detection effectiveness was not reported. For the non-industrial case studies, the reduction in fault detection effectiveness reported is significantly lower than in this case study, even when the reported reduction in test suite size was comparable with the reduction reported in this study. However, in these studies, either the faults were seeded [11] or the versions of the system were created artificially [10, 38].

No industrial studies that reported results for the hybrid technique were found. The non-industrial studies reported results that are very similar to the results reported by this study.

8. CONCLUSION

This paper presented a case study of the effect of test case prioritization on fault detection rates and the effect of test case selection, test suite minimization and a hybrid approach on the reduction in test suite size and the reduction in fault finding effectiveness on a non-trivial, real-world industrial system. The study used real successive versions of a software system containing real regression faults, and test suites created using a systematic state-of-the-art approach. As discussed in the related work section, such studies are extremely rare, despite the large body of work that exists.

Four different coverage-based prioritization techniques, based on five coverage criteria, were applied to four versions of the system. Test case prioritization was also compared to random ordering and the optimal order to investigate both the practical gains and room of improvement of coverage-based prioritization techniques. A coverage-based test case selection technique, test suite minimization technique and a hybrid technique, based on the five coverage criteria, were also applied to the four versions of the system to investigate the benefits and trade-offs between execution cost and fault detection effectiveness involved in using such techniques.

The study showed that prioritization techniques based on additional coverage using finer grained coverage criteria (block, basic block and decision) outperformed all other techniques used in the study, including random ordering (APFD = 74.2 vs 59.6 to 70.8 for the other techniques). Consistent with a previously reported study, techniques using modification information did not show significant improvements over techniques that only rely on overall coverage information.

The results of this study, when looking at all reported studies on prioritization, show that industry studies yield less positive results regarding the effectiveness of coverage-based techniques than artificial ones based on smaller programs and seeded faults. This highlights once again the need to perform industrial case studies, which are still far too few.

There is still substantial room for improvement in prioritization techniques, as shown by the comparison to the optimal order, and more research is required in this direction. It is, however, unlikely that relying on other coverage criteria and finer grained source code modification analysis will yield significant improvements. Novel ways of looking at the problem are required.

The study and the comparison to related work showed that the effectiveness of test selection techniques and their ability to reduce execution costs are likely to be dependent on the location and size of the modifications to the system. Test engineers could perform a simple analysis of the changes to the system to decide if using test case selection techniques would be beneficial.

The results also revealed that using test suite minimization techniques with finer grained coverage criteria provides a trade-off between the reduction in execution cost (78.5–79.5%) and the reduction in fault finding effectiveness (25.4–29.7%) that might be suitable in certain contexts in non-critical systems, where testing time and resources are limited.

Surprisingly, there are no industrial case studies that report on the impact of test suite minimization on fault detection effectiveness. On the other hand, non-industrial case studies results showed reductions in fault detection effectiveness that range from 2% to 60%.

Finally, a hybrid approach that combines test case selection and test suite minimization did not provide a significant improvement compared with test suite minimization techniques.

Many more industrial studies are needed where the strategy used to form the test suite, the nature and extent of the changes to the system, and information about the detectability of regression faults are clearly reported so that differences in studies can be clearly explained. Otherwise, it will be very hard to determine whether, and under which conditions, such techniques are beneficial. This also implies that artificial studies where faults are seeded, test suites are not formed according to any approach used in practice, and versions do not match real releases are unlikely to yield any useful result, given the impact of such factors on results.

ACKNOWLEDGEMENTS

Supported by the Fonds National de la Recherche, Luxembourg (FNR/P10/03 and FNR 4082113) and NSERC discovery grants (Canada).

REFERENCES

1. Di Nardo D, Alshahwan N, Briand L, Labiche Y. Coverage-based test case prioritisation: An industrial case study. *Proceedings of the 6th IEEE International Conference on Software Testing, Verification and Validation (ICST '13)*, IEEE: Luxembourg, 2013; 302–311.
2. Yoo S, Harman M. Regression testing minimisation, selection and prioritisation: a survey. *Software Testing, Verification and Reliability (STVR)* 2012; **22**(2):67–120.
3. Rothermel G, Harrold MJ. Analyzing regression test selection techniques. *IEEE Transactions on Software Engineering (TSE)* 1996; **22**(8):529–551.
4. Rothermel G, Harrold MJ. Empirical studies of a safe regression test selection technique. *IEEE Transactions on Software Engineering (TSE)* 1998; **24**(6):401–419.
5. Rothermel G, Harrold MJ, Dedhia J. Regression test selection for C++ software. *Software Testing, Verification and Reliability (STVR)* 2000; **10**(2):77–109.
6. Graves TL, Harrold MJ, Kim JM, Porter A, Rothermel G. An empirical study of regression test selection techniques. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 2001; **10**(2):184–208.
7. Harrold MJ, Jones JA, Li T, Liang D, Orso A, Pennings M, Sinha S, Spoon SA, Gujarathi A. Regression test selection for Java software. *Proceedings of the 16th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '01)*, ACM: Tampa, FL, USA, 2001; 312–326.
8. Harrold MJ. Testing evolving software. *Journal of Systems and Software* 1999; **47**(2-3):173–181.
9. Rothermel Gregg, Elbaum Sebastian, Malishevsky Alexey G., Kallakuri Praveen, Qiu Xuemei. On test suite composition and cost-effective regression testing. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 2004; **13**(3):277–331.
10. Wong WE, Horgan JR, Mathur AP, Pasquini A. Test set size minimization and fault detection effectiveness: A case study in a space application. *Journal of Systems and Software* 1999; **48**(2):79–89.
11. Elbaum S, Kallakuri P, Malishevsky A, Rothermel G, Kanduri S. Understanding the effects of changes on the cost-effectiveness of regression testing techniques. *Software Testing, Verification and Reliability (STVR)* 2003; **13**(2):65–83.
12. Sampath Sreedevi, Bryce Renée C. Improving the effectiveness of test suite reduction for user-session-based testing of web applications. *Information and Software Technology* 2012; **54**(7):724–738.
13. Do H, Rothermel G. A controlled experiment assessing test case prioritization techniques via mutation faults. *Proceedings of the 21st IEEE International Conference on Software Maintenance (ICS '05)*, pages 411–420, IEEE: Budapest, Hungary, 2005; 411–420.
14. Do H, Rothermel G, Kinneer A. Empirical studies of test case prioritization in a JUnit testing environment. *Proceedings of the 15th International Symposium on Software Reliability Engineering (ISSRE '04)*, 2004; 113–124.
15. Elbaum S, Malishevsky AG, Rothermel G. Test case prioritization: a family of empirical studies. *Transactions on Software Engineering (TSE)* 2002; **28**(2):159–182.
16. Elbaum S, Rothermel G, Kanduri S, Malishevsky AG. Selecting a cost-effective test case prioritization technique. *Software Quality Control* 2004; **12**(3):185–210.
17. Rothermel G, Untch RJ, Chu C. Prioritizing test cases for regression testing. *Transactions on Software Engineering (TSE)* 2001; **27**(10):929–948.
18. Telcordia Technologies. *Telcordia Software Visualization and Analysis Toolsuite – User Manual*, 1998. (Available from: <ftp://ftp.software.ibm.com/s390/mvs/info/test/xsuds.ps>) [last accessed 4 April 2014].
19. Wong WE, Horgan JR, London S, Mathur AP. Effect of test set minimization on fault detection effectiveness. *Software—Practice & Experience* 1998; **28**(4):347–369.
20. Ostrand TJ, Balcer MJ. The category-partition method for specifying and generating functional tests. *Communications of the ACM* 1988; **31**(6):676–686.
21. Marick B. *The Craft of Software Testing: Subsystems Testing Including Object-Based and Object-Oriented Testing*. Pearson Education, 1995.
22. Do H, Elbaum S, Rothermel G. Infrastructure support for controlled experimentation with software testing and regression testing techniques. *Proceedings of the 2004 International Symposium on Empirical Software Engineering (ISESE '04)*, IEEE: Redondo Beach, California, 2004; 60–70.
23. Elbaum S, Malishevsky A, Rothermel G. Incorporating varying test costs and fault severities into test case prioritization. *Proceedings of the 23rd International Conference on Software Engineering (ICSE '01)*, IEEE: Toronto, Ontario, Canada, 2001; 329–338.
24. Rothermel G, Harrold MJ, Ostrin J, Hong C. An empirical study of the effects of minimization on the fault detection capabilities of test suites. *Proceedings of the International Conference on Software Maintenance (ICSM '98)*, IEEE: Bethesda, MD, 1998; 34–43.
25. Arcuri A, Briand L. A practical guide for using statistical tests to assess randomized algorithms in software engineering. *Proceedings of the 33rd International Conference on Software Engineering (ICSE 11)*, IEEE: Honolulu, HI, 2011; 1–10.
26. Alshahwan N, Harman M. Augmenting test suites effectiveness by increasing output diversity (NIER track). *Proceedings of the 34th International Conference on Software Engineering - New Ideas and Emerging Results Track (ICSE NIER 12)*, IEEE: Zurich, 2012; 1345–1348.
27. Andrews JH, Briand LC, Labiche Y, Namin AS. Using mutation analysis for assessing and comparing testing coverage criteria. *IEEE Transactions on Software Engineering (TSE)* 2006; **32**(8):608–624.

28. Hutchins M, Foster H, Goradia T, Ostrand T. Experiments of the effectiveness of dataflow- and controlflow-based test adequacy criteria. *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, IEEE: Sorrento, Italy, 1994; 191–200.
29. Namin AS, Andrews JH. The influence of size and coverage on test suite effectiveness. *Proceedings of the 18th International Symposium on Software Testing and Analysis (ISSTA '09)*, ACM: Chicago, Illinois, USA, 2009; 57–68.
30. Wohlin C, Runeson P, Host M, Ohlsson MC, Regnell B, Wesslen A. *Experimentation in Software Engineering: An Introduction*. Kluwer Academic: Norwell, Massachusetts, U.S.A., 2000.
31. Srivastava A, Thiagarajan J. Effectively prioritizing tests in development environment. *Proceedings of the 2002 ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '02)*, ACM: Rome, Italy, 2002; 97–106.
32. Carlson R, Do H, Denton A. A clustering approach to improving test case prioritization: An industrial case study. *Proceedings of the 2011 27th IEEE International Conference on Software Maintenance (ICSM '11)*, 2011; 382–391.
33. Engström E, Runeson P, Ljung A. Improving regression testing transparency and efficiency with history-based prioritization – an industrial case study. *Proceedings of the 4th IEEE International Conference on Software Testing, Verification and Validation (ICST '11)*, IEEE: Berlin, 2011; 367–376.
34. Sherriff M, Lake M, Williams L. Prioritization of regression tests using singular value decomposition with empirical change records. *Proceedings of the 18th IEEE International Symposium on Software Reliability (ISSRE '07)*, IEEE: Trollhättan, Sweden, 2007; 81–90.
35. Srikanth H, Banerjee S. Improving test efficiency through system test prioritization. *Journal of Systems and Software* 2012; **85**(5):1176–1187.
36. Srikanth H, Cohen MB. Regression testing in software as a service: An industrial case study. *Proceedings of the 2011 27th IEEE International Conference on Software Maintenance (ICSM '11)*, IEEE: Williamsburg, VI, 2011; 372–381.
37. Hsu HY, Orso A. MINTS: A general framework and tool for supporting test-suite minimization. *Proceedings of the 31st International Conference on Software Engineering (ICSE '09)*, IEEE: Vancouver, BC, 2009; 419–429.
38. Rothermel G, Harrold MJ, von Ronne J, Hong C. Empirical studies of test-suite reduction. *Software Testing, Verification and Reliability (STVR)* 2002; **12**(4):219–249.